Report - Assignment 3

All numbers below come from results/results.csv (seed new Random(42), reproducible with one command, see README). Notation: V = n vertices, E = m edges. The program reproduces all ten reference values exactly (MST weight, SCC count, sum of DAG distances); the benchmark throws an exception if one of them, the Kruskal = Prim agreement, a topological order, or the exact scan counts (Kosaraju 2m, Kahn m, DAG relaxation m) is wrong.

n / m	MST (Kruskal = Prim)	SCC count	Sum of DAG distances
1 000 / 4 000	147 970	27	396 274
1 000 / 20 000	30 363	1	252 411
3 162 / 12 648	468 911	137	1 271 911
3 162 / 63 240	97 589	1	723 157
10 000 / 40 000	1 486 139	373	3 941 828
10 000 / 200 000	309 550	1	2 381 342
31 623 / 126 492	4 715 515	1 188	12 551 891
31 623 / 632 460	968 517	1	7 376 612
100 000 / 400 000	15 012 094	3 923	40 096 539
100 000 / 2 000 000	3 056 681	1	23 512 087

1. Complexity table

Algorithm	Time	Extra space	Justification
DSU find / union	O(alpha(n)) amortized	O(n)	union by rank keeps trees O(log n) high, path compression flattens them; together a sequence of k operations costs O(k alpha(n)), and alpha(n) <= 4 for any realistic n
Kruskal	O(E log E) = O(E log V)	O(E + V)	merge sort of E edges dominates; the loop does <= 2E finds + V-1 unions at near-constant cost each; index array + merge buffer + DSU
Prim (lazy, binary heap)	O(E log E) = O(E log V)	O(V + E)	every arc is scanned once (2E) and pushed at most once; each

			push/pop costs O(log heap size) <= O(log E); heap holds up to 2E entries
Kosaraju	O(V + E)	O(V + E)	two DFS passes, each arc scanned once per pass; the reversed graph needs O(V + E); explicit stack O(V)
Kahn	O(V + E)	O(V)	in-degree array + queue, each arc decrements one counter
DAG shortest paths	O(V + E)	O(V)	one pass over the topological order, every arc relaxed exactly once (the topological order itself costs another O(V + E)); dist[], parent[]

2. Counters vs formulas

If an algorithm really has the stated complexity, the measured counter divided by the formula must stay (nearly) constant while n grows 100 times.

Sparse graphs (m = 4n)

algorithm	ratio	n=1000	n=3162	n=10000	n=31623	n=100000
Kruskal	dsuOps / edgeScans	2.324	2.283	2.259	2.252	2.250
Kruskal	dsuOps / E	1.791	2.019	2.182	2.232	2.248
Kruskal	comparisons / (E log2 E)	0.896	0.928	0.940	0.926	0.949
Prim	edgeScans / E	2.000	2.000	2.000	2.000	2.000
Prim	comparisons / (E log2 V)	2.195	2.188	2.186	2.188	2.180
Kosaraju	edgeScans / (V+E)	1.600	1.600	1.600	1.600	1.600
Kahn	edgeScans / E	1.000	1.000	1.000	1.000	1.000
DagShortestPath	edgeScans / E	1.000	1.000	1.000	1.000	1.000

Dense graphs (m = 20n)

algorithm	ratio	n=1000	n=3162	n=10000	n=31623	n=100000
Kruskal	dsuOps / edgeScans	2.232	2.219	2.241	2.166	2.134
Kruskal	dsuOps / E	0.480	0.506	0.465	0.654	0.794
Kruskal	comparisons / (E log2 E)	0.935	0.922	0.946	0.953	0.942
Prim	edgeScans / E	2.000	2.000	2.000	2.000	2.000
Prim	comparisons / (E log2 V)	2.617	2.536	2.475	2.424	2.384
Kosaraju	edgeScans / (V+E)	1.905	1.905	1.905	1.905	1.905
Kahn	edgeScans / E	1.000	1.000	1.000	1.000	1.000
DagShortestPath	edgeScans / E	1.000	1.000	1.000	1.000	1.000

Reading the tables:

- **Traversals are exactly linear.** Prim (2E), Kosaraju (2E), Kahn (E) and DAG relaxation (E) have ratios that do not move at all. Kosaraju / (V+E) is 1.600 on sparse and 1.905 on dense graphs only because 2E/(V+E) depends on the density, not on n.
- **Kruskal sorting** follows E log2 E: comparisons / (E log2 E) stays between 0.90 and 0.95 over a 100-fold growth.
- **Kruskal and DSU.** dsuOps / E drifts on dense graphs (0.48 -> 0.79) not because DSU gets slower, but because Kruskal stops as soon as the tree is complete and examines a growing share of the edges (see section 4). Normalised by the edges actually examined, the DSU cost is flat: dsuOps / edgeScans = 2.32, 2.28, 2.26, 2.25, 2.25 on sparse graphs (two finds per examined edge plus the successful unions, about 2.25 in total); on dense graphs it is the same constant within a few percent (2.23, 2.22, 2.24, 2.17, 2.13). That is the almost-constant alpha(n) cost of DSU in practice.
- **Prim** pushes into the heap about 2.2 - 2.6 comparisons per E log2 V; the ratio is flat on sparse and slowly decreasing on dense graphs (many arcs lead to vertices that are already in the tree and are never pushed), i.e. it is bounded by the O(E log V) formula.

3. Slope check (log-log, least squares over the five sizes)

algorithm	edgeScans slope, sparse	edgeScans slope, dense
Prim	1.000	1.000
Kosaraju	1.000	1.000
Kahn	1.000	1.000
DagShortestPath	1.000	1.000

Kruskal	1.055	1.119

Total counted operations (dsuOps + edgeScans + comparisons): Kosaraju, Kahn and DagShortestPath 1.000; Kruskal 1.096 / 1.088; Prim 1.102 / 1.085 (sparse / dense).
Slope of comparisons alone: Kruskal 1.105 / 1.087, Prim 1.109 / 1.090.

All traversals give 1.00 - linear work. Kruskal is slightly steeper. The log factor of E log E is clearly visible in comparisons (slope about 1.1 = 1 + a small log term over this range). For Kruskal's edgeScans the extra steepness has a second cause: the number of edges it must look at before the tree is connected grows faster than n (section 4).

4. Correctness arguments

Cut property (MST). Let (S, V \ S) be any cut of the graph that no edge of an already-chosen edge set A crosses, and let e be a lightest edge crossing it. Then some minimum spanning tree contains A and e. Proof sketch: take an MST T that contains A. If e is not in T, adding e to T creates a cycle; this cycle crosses the cut at least twice, so it contains another edge f crossing the cut, and f is not in A (A does not cross the cut). Since e is lightest across the cut, w(e) <= w(f), so T - f + e is again a spanning tree, with weight not larger, still containing A and e. Hence choosing e is always safe. **Kruskal** uses the cut property with S = one component of the current forest: the lightest remaining edge that joins two different components (find(u) != find(v)) is the lightest edge across the cut around its component, so it is safe; an edge inside a component would close a cycle and is skipped. **Prim** uses it with S = the set of vertices already in the tree: the heap returns the lightest edge leaving S, which is safe; stale heap entries pointing into S are skipped. Both algorithms therefore build a tree that is optimal at every step, so their total weights are equal (the trees themselves may differ when weights tie) - the benchmark checks this on all ten graphs.

Induction on the topological order (DAG shortest paths). Let v_1, ..., v_n be the topological order and d*(v) the true shortest distance from the virtual source (sources start at 0). Claim: when the algorithm is about to process v_i, dist[v_i] = d*(v_i). Base: v_1 has no incoming edges (any edge into it would have to come from an earlier vertex), so it is a source and dist = 0 = d*. Step: assume the claim for v_1..v_{i-1}. Every shortest path to v_i ends with an edge (v_j, v_i) with j < i, or v_i is a source. By the induction hypothesis dist[v_j] = d*(v_j) was final when v_j was processed, and processing v_j relaxed the edge (v_j, v_i), so dist[v_i] <= d*(v_j) + w(v_j, v_i) = d*(v_i); dist never goes below the length of a real path, so equality holds. Each edge is relaxed once, in one pass of O(V + E).

Why negative weights are safe in a DAG but not in a general graph. The induction above never uses the sign of the weights - it only uses that all predecessors of a vertex come before it. A vertex is final when it is reached in the order, not when it is the "closest" unprocessed vertex. Dijkstra needs the second property (a vertex taken from the queue cannot be improved later),

and that breaks with a negative edge. In a general graph a negative cycle makes shortest distances undefined (-infinity), and even without one a single pass in any fixed order is not enough (Bellman-Ford needs V-1 passes, O(VE)). A DAG has no cycles at all, so no negative cycle can exist and one pass suffices. (The tests compare against Bellman-Ford on random DAGs with weights in [-10, 20].)

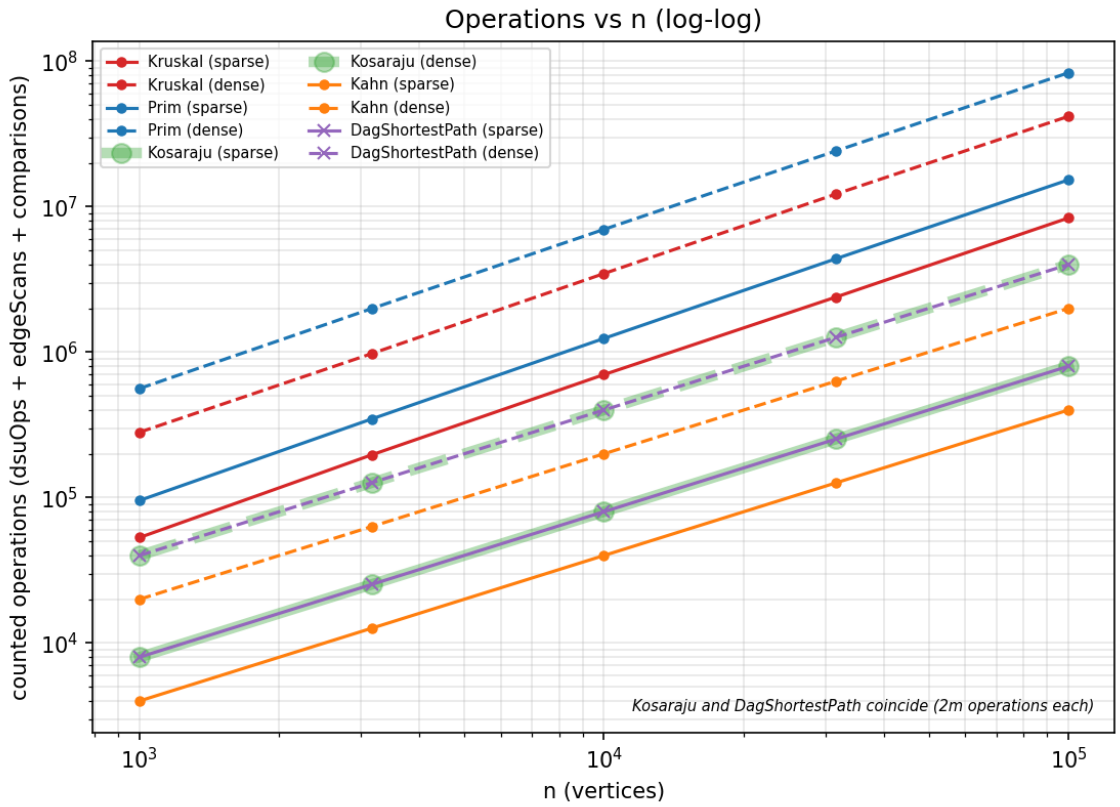
5. Which algorithm when

Situation	Choice	Why
Sparse undirected graph, edge list	Kruskal	O(E log E) with E = O(V) is cheap; needs no adjacency structure; DSU costs about 2.25 operations per examined edge. Measured: about 1.8x fewer total counted operations than Prim (8.4e6 vs 1.5e7 at n = 100 000) and 3.4x less wall-clock time (54 vs 186 ms; time is only a hint)
Sparse undirected graph, adjacency list	Prim or Kruskal	same O(E log V); Kruskal needs the edge list (we keep both); Prim works directly on the adjacency list
Dense undirected graph, adjacency list / matrix	In theory Prim with an indexed (decrease-key) heap or the O(V^2) array version. In our lazy-heap Prim, Kruskal still wins	our measurements: on dense graphs Kruskal examines only 21 - 37 % of the sorted edges and uses half of Prim's counted operations (4.2e7 vs 8.3e7 at n = 100 000; ratio 1.98 - 2.04 at every size), because lazy Prim scans all 2E arcs and pays heap work for them. So the textbook rule "Prim for dense graphs" holds only for the better Prim variants.
Edges arrive one by one / "are u and v connected now?"	DSU (Kruskal-style)	union-find answers in nearly constant time without rebuilding anything
Directed graph, cycles possible, need ordering	Kahn	detects a cycle (fewer than n vertices leave the queue) instead of returning a wrong order; O(V + E)
Directed graph, need the cyclic groups (deadlocks)	Kosaraju	O(V + E); simple to explain; needs the reversed graph

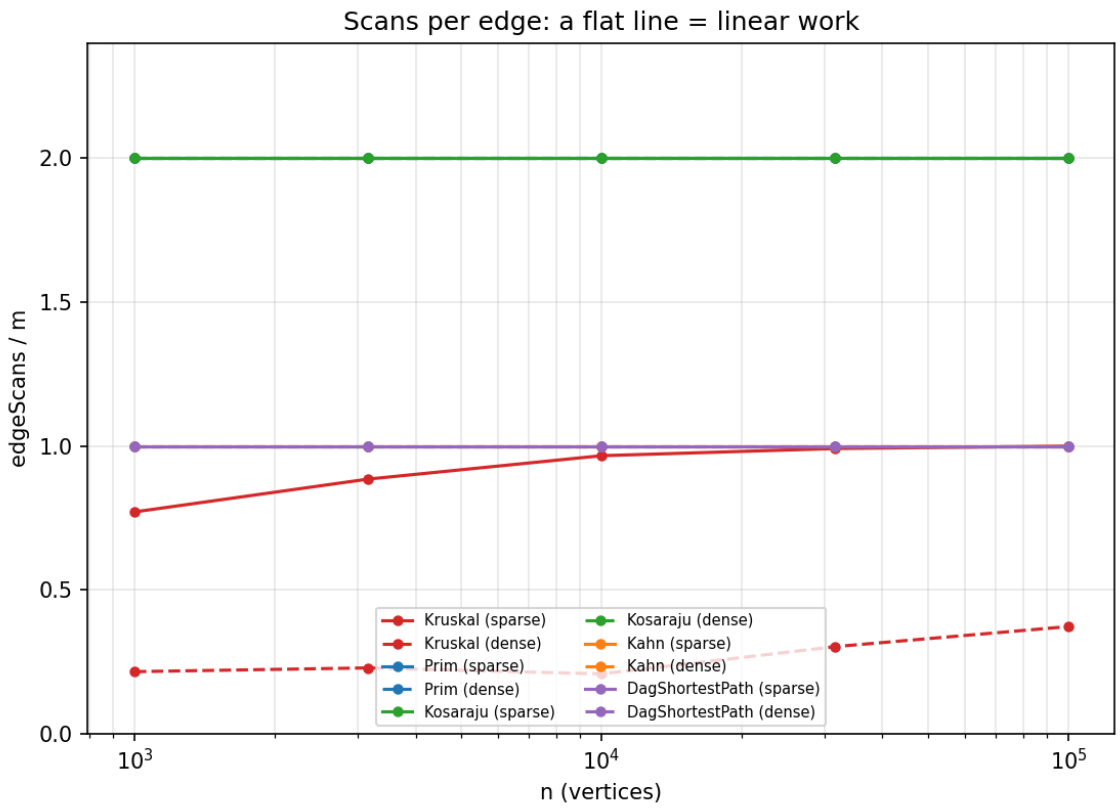
DAG, shortest paths, even with negative weights	topological order + one relaxation pass	O(V + E), exactly m relaxations; Dijkstra is invalid with negative edges, Bellman-Ford costs O(VE)
Graph with cycles and non-negative weights	Dijkstra	DAG relaxation is not applicable (not part of this assignment)
Graph with cycles and negative weights	Bellman-Ford	the only one of these that detects negative cycles

6. Plots

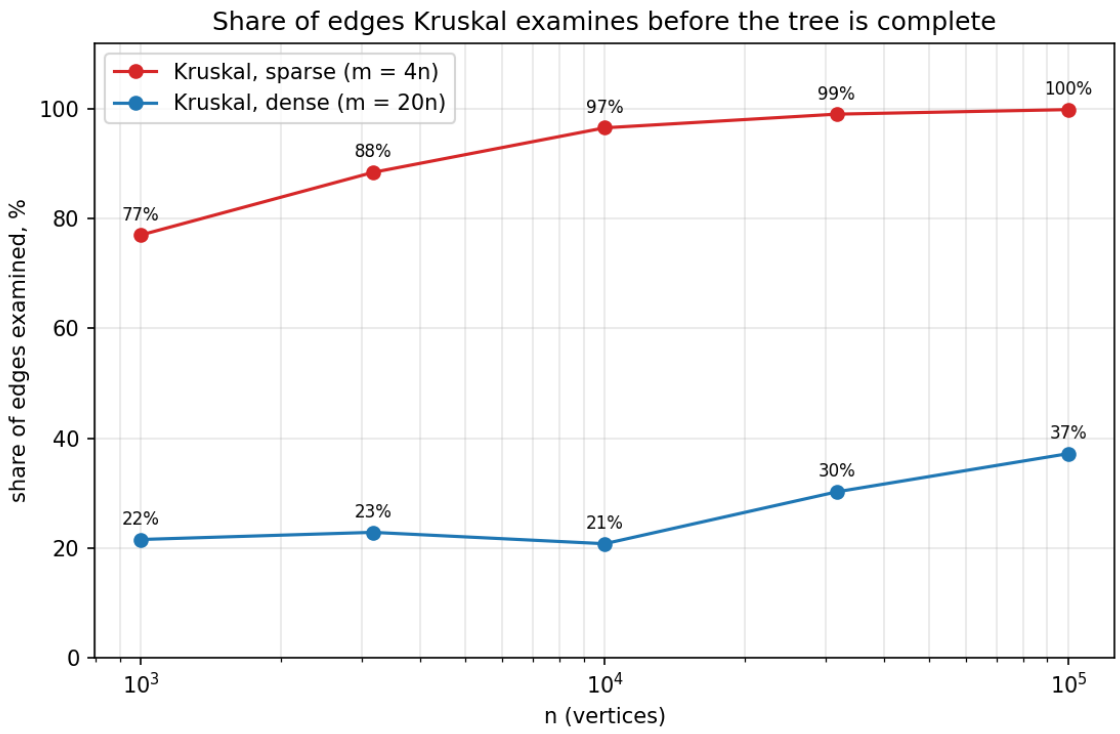
Operations vs n (log-log). Parallel straight lines of slope about 1; lines are ordered by density (m = 20n above m = 4n). Kosaraju and DagShortestPath coincide (2m operations each).



Scans per edge (edgeScans / m). Flat lines at 2 (Prim, Kosaraju), 1 (Kahn, DagShortestPath) - linear work. Only Kruskal rises.



Share of edges Kruskal examines. On sparse graphs (m = 4n) Kruskal needs almost every edge (77 % -> 99.9 %) because the last edges of the sorted list are needed to connect the final vertices; the share grows with n. On dense graphs (m = 20n) it stops after 21 % - 37 % of the edges. This is where each MST algorithm wins: Kruskal's early stop pays off on dense graphs, while on sparse graphs there is nothing to skip. The growth with n is consistent with the connectivity threshold of random graphs (a random edge set connects n vertices only after about (n ln n)/2 edges): for m = 20n the predicted share ln(n)/40 is 17 % at n = 1 000 and 29 % at n = 100 000 (measured 21 % and 37 % - the same trend, rough agreement).



7. Time

Median-of-5 times are in results.csv. For example at n = 100 000 / m = 2 000 000: Kruskal 300 ms, Prim 1403 ms, Kosaraju 440 ms, Kahn 152 ms, DagShortestPath 385 ms; at m = 400 000: Kruskal 54 ms, Prim 186 ms, Kosaraju 35 ms, Kahn 9 ms, DagShortestPath 22 ms. The times are not the main evidence: they depend on the CPU, the JVM and JIT warm-up (the two smallest graphs are mostly noise: 0.2 - 7 ms), garbage collection and background load (measured in a container with 1 core and 3 GB of memory). They also show effects that no complexity formula has: Kosaraju takes 440 ms at n = 100 000 vs 85 ms at n = 31 623 although its counter grew only 3.2 times, because the randomly accessed arrays no longer fit into the CPU cache. The counters (e.g. Kosaraju = 2m, Kahn = m, Kruskal comparisons = 0.93 E log2 E) are identical on every machine and are what the conclusions in this report are based on.